GitHub Copilot 워크플로 과정

직접 만들고 공유하기

오늘 배운 것 하나를 골라 내 일에 붙여 보고, 서로 보여 줍니다

1. 복기
2. 주제 고르기
3. 만들기
4. 공유
5. 만족도 조사

복기 — 오늘 손에 쥔 도구

M2

instructions

instructions 40줄 · `applyTo` · 경로별

M3

Skill · Agent

반복 절차는 Skill, 역할은 커스텀 에이전트

M4

사용량 · Harness

새 대화 · `#file` 로 낭비 줄이기, Hooks 로 차단 · 자동 테스트

M5

MCP · 보안

GitHub · Atlassian · 사내 MCP, 인젝션 방어

M6

PR · 리뷰 · CI/CD

PR 템플릿 · Copilot 리뷰 3분류 · Actions 로 배포 · Jira Done

공통

사람이 판단

만들기는 Copilot, 확인 · 승인 · 머지는 사람

01

주제 고르기

남은 시간 안에 끝나는 크기로, 화면에서 결과를 보여 줄 수 있는 것 하나

세 갈래 중 하나 — 고르기 어려우면 A

A · 내 업무

내 repo 에 붙이기

평소 쓰는 저장소에 instructions 40줄 + 가장 반복되는 작업 Skill 하나 — 실제로 한 번 돌려 보기

B · equip-monitor

새 기능 한 바퀴

요구사항 하나를 `/req-to-jira` → `/jira-implement` → `/pr-create` → CI · 리뷰까지

C · 자동화

도구 하나 더 만들기

커스텀 에이전트 · hook · Skill · Actions 워크플로 중 하나를 새로

사내 코드는 사내 정책 안에서 — 토큰 · 비밀번호 · 고객 데이터는 프롬프트와 공유 화면에 넣지 않습니다

주제 예시

갈래	예시	참고 모듈
A	우리 팀 코딩 규약을 instructions 40줄로 · 커밋 메시지 규칙을 `applyTo` instructions 로	M2
A	매주 하는 배포 점검 · 로그 정리를 Skill 로 (검증 스크립트 포함)	M3
B	알람 CSV 내보내기 · 장비별 통계 API · 임계값 설정 화면	M6
C	읽기 전용 `reviewer` 에이전트 — 우리 팀 리뷰 관점 5개	M3
C	`git push --force` · 비밀 파일 수정을 막는 hook	M4
C	PR 마다 lint · 테스트를 돌리는 워크플로	M6

만들 때 지킬 것 네 가지

1. **작게**
   남은 시간 안에 한 번 돌려 볼 수 있는 크기 — 커지면 범위를 줄이기

2. **검증과 함께**
   테스트 · 실행 결과 · 스크립트 출력처럼 됐다는 증거를 남기기

3. **사람이 판단**
   Copilot 의 제안은 읽고 승인 — 쓰기 카드 · 커밋 · push 는 내가 확인한 뒤

4. **기록하며**
   쓴 프롬프트 · 막힌 곳 · 고친 방법을 메모 — 그대로 공유 자료가 됩니다

실습

만들기

고른 주제 하나를 Copilot 과 함께 만들고,
한 번 실행해 결과를 확인한다

준비

- 주제 · 대상 저장소 정하기 (A 내 repo / B equip-monitor `main` / C 둘 중 하나)
- 새 대화(하네스 **Local**) — 지난 대화 이어 쓰지 않기
- 메모장: 프롬프트 · 막힌 곳 · 결과

실습 1

계획부터 — 바로 만들지 않기

1. 오른쪽 프롬프트에 주제 한 줄
2. Copilot 이 만든 계획 · 파일 목록 · 확인 방법을 읽기
3. 남은 시간보다 크면 범위를 줄여 다시

프롬프트 / 명령

```
# 계획만 (아직 만들지 말 것)
오늘 과정에서 배운 <instructions / Skill / 에이
전트 / hook / MCP / 워크플로> 로
<만들 것 한 줄> 을 만들려고 한다.
<남은 시간>분 안에 끝나도록 범위를 정하고,
1) 만들 파일 목록 2) 단계 3) 확인 방법
을 표로 보여 줘. 파일은 아직 만들지 마.
```

✅ **계획이 남은 시간 크기인가 · 확인 방법이 실행 가능한가**

실습 2

만들고 · 돌려 보고 · 고치기

1. 계획대로 만들기 — 파일마다 Keep 전에 읽기
2. 확인 방법대로 한 번 실행 — 테스트 · 명령 · 화면 중 하나
3. 실패하면 우회 말고 원인을 묻고 고치기
4. 됐으면 커밋 · push — 제출은 GitHub 링크로

프롬프트 / 명령

```
# 만들기
위 계획대로 만들어. 파일 하나 끝날 때마다 멈추고 무엇을 했는지 한 줄로 알려 줘.

# 확인
계획의 확인 방법대로 실행하고 결과를 그대로 보여 줘.

# 실패했을 때
실패 원인을 먼저 설명해. 테스트를 끄거나 건너뛰는 우회는 하지 마.
```

✅ **확인 방법을 실제로 한 번 돌렸다 · push 해서 GitHub 링크가 있다**

실습 3

공유 카드 만들기 — Copilot 에게 정리 맡기기

1. 오른쪽 프롬프트로 공유 카드 초안
2. 메모와 비교해 사실만 남기기
3. 보여 줄 화면 · 파일을 열어 두기

프롬프트 / 명령

```
# 공유 카드 (3분 발표용)
이 대화에서 한 일을 아래 다섯 줄로 정리해.
1) 풀려던 문제
2) 쓴 Copilot 기능 (instructions / Skill / 에이전트 / MCP / hook / 워크플로)
3) 핵심 프롬프트 한 줄
4) 결과 – 무엇이 되고 무엇이 안 됐나
5) 막힌 곳과 다음에 해 볼 것
```

✅ **다섯 줄이 사실과 맞는가 — 안 된 것도 그대로**

02

공유하기

한 사람 3분 — 문제, 쓴 기능, 결과 화면, 막힌 곳

3분 공유 순서

순서	보여 줄 것	시간
① **문제**	무엇을 줄이거나 자동화하고 싶었나	30초
② **쓴 기능**	instructions · Skill · 에이전트 · MCP · hook · 워크플로 중 무엇을 왜	30초
③ **결과 화면**	파일 · 실행 결과 · PR · 테스트 — 실제 화면으로	1분 30초
④ **막힌 곳**	어디서 막혔고 어떻게 풀었나 (못 풀었으면 그대로)	30초

실습 4

공유 — 한 사람 3분

1. 발표자: 공유 카드 다섯 줄 + 결과 화면
2. 듣는 사람: 가져갈 것 한 가지 메모
3. 인원이 많으면 조별 공유 → 조마다 대표 1명

메모 양식

```
# 듣는 사람 메모
발표자 · 주제:
쓴 기능:
우리 팀에 가져갈 것 한 가지:
```

만든 결과물 제출

오늘 만든 결과물은 GitHub 링크로 아래 폼에 제출해 주세요

https://forms.gle/UjpqcT87QHYFYMys8

- 어떤 형태든 GitHub 링크 하나 — 저장소 · 브랜치 · PR · 커밋 · 파일 링크 모두 가능
- 비공개 저장소면 링크가 열리는지 확인 (HWM-RND 조직 저장소면 그대로)
- 덜 끝났어도 지금 상태를 push 하고 그 링크로 제출

03

마무리

오늘 하루를 정리하고, 사후 만족도 조사로 마칩니다

핵심 요약

1. instructions · Skill · 에이전트 · MCP · hook · 워크플로 — 하나라도 내 저장소에 붙이면 시작
2. 작게 만들고, 검증하고, 사람이 판단 — 오늘 실습 전체에 흐른 원칙
3. 막힌 곳이 가장 쓸모 있는 공유 — 팀에도 그대로 나누기
4. 만든 결과물은 GitHub 링크로 제출 폼(forms.gle/UjpqcT87QHYFYMys8)에
5. 만족도 조사 QR — 다음 과정을 더 낫게 만드는 데 씁니다

사후 만족도 조사



1. **휴대폰 카메라로 QR 스캔**
   2~3분이면 끝납니다

2. **좋았던 것 · 아쉬웠던 것**
   다음 과정에 그대로 반영합니다

3. **만든 결과물은 따로**
   앞의 제출 폼에 GitHub 링크로